INTERVIEWER PERSONALISED AVAILABILITY

Oracle database design, preference rules, test data, SQL queries, and duration-based slot generation

Objective
Store recurring weekly preferences and use them to generate valid interview start times for a selected date. Focus is limited to personalised preference slots and their evaluation.

Design at a glance

Area	Decision
Existing mapping	Reuse the existing interviewer mapping table for interviewer SID, Oracle candidate ID, and time zone.
New table	Create one table: INTERVIEWER_PREFERENCE_SLOT.
Preference flag	Use PREFERENCE_ENABLED in the parent table if an equivalent flag does not already exist.
Availability flag	IS_AVAILABLE NUMBER(1): 1 = allowed interval; 0 = blocked interval.
Time model	Store recurring weekday and local minutes after midnight.
Duration handling	Generate candidate start/end intervals dynamically for 30, 45, 60, or 120-minute interviews.

Implementation assumptions
INTERVIEWER_MASTER is a placeholder name. Replace it with the actual table. The pair (INTERVIEWER_SID, ORACLE_CAND_ID) must be a primary key or unique key in the parent table. Child column types and lengths must match exactly.

Contents: flow and rules; table design; DDL; test schedules; SELECT queries; preference evaluation; duration-based start generation; Java integration guidance; test checklist.

1. Personalised-slot flow

The preference layer determines which interview intervals are permitted by the interviewer. The application can use these preference-eligible intervals to populate the selected-date calendar.

1	Interviewer saves weekly preferences Choose weekdays, local start/end times, and any unavailable exceptions.
2	Validate and persist Validate times and weekdays; save all slot rows in one transaction; enable preferences after a successful save.
3	Candidate selects a date and duration Use the interviewer mapping to retrieve the interviewer SID, Oracle candidate ID, and time zone.
4	Read preference configuration Check PREFERENCE_ENABLED and load the saved recurring rows for the interviewer mapping.
5	Generate candidate intervals For every AVAILABLE window, generate start = windowStart + n × step and end = start + duration.
6	Apply blocked exceptions Discard any candidate interval that overlaps an UNAVAILABLE window.
7	Return the personalised starts Display only intervals that fit completely inside an allowed window and pass the blocked-window test.
8	Revalidate on selection Re-read the latest preference rows before accepting the selected start time.

Preference evaluation rules

1. When preferences are enabled, AVAILABLE rows form an allowlist. A day/time not covered by an AVAILABLE row is not allowed. 2. Any overlapping UNAVAILABLE row overrides an AVAILABLE row. 3. The full interview duration must fit inside one AVAILABLE window. 4. Evaluate weekday and minute-of-day in the interviewer time zone. 5. Intervals use start-inclusive/end-exclusive logic: [start, end).

Configuration rule
If PREFERENCE_ENABLED = Y but there are no rows, treat it as inconsistent configuration. Do not interpret this as all times being allowed. If preferences are disabled, this document returns no personalised restrictions; the calling application decides its separate behaviour.

2. Table design and time encoding

Use a headerless slot table: each row represents one recurring weekly interval associated with the interviewer mapping in the parent table.

Column	Type	Purpose
SLOT_ID	NUMBER identity	Primary key generated by Oracle.
INTERVIEWER_SID	Same as parent	Interviewer SID; part of the composite foreign key.
ORACLE_CAND_ID	Same as parent	Secondary interviewer-mapping identifier; part of the composite foreign key. Confirm its meaning in your schema.
DAY_OF_WEEK	NUMBER(1)	Monday=1, Tuesday=2, Wednesday=3, Thursday=4, Friday=5, Saturday=6, Sunday=7.
START_MINUTE	NUMBER(4)	Local start time as minutes after midnight; 0-1439.
END_MINUTE	NUMBER(4)	Local end time as minutes after midnight; 1-1440.
IS_AVAILABLE	NUMBER(1)	1 = allowed; 0 = blocked.
CREATED_AT	TIMESTAMP	Defaults to SYSTIMESTAMP at insert.
UPDATED_AT	TIMESTAMP	Default at insert; refresh in a trigger or application update.

Time conversion reference

Local time	Minute value	Local time	Minute value
12:00 AM / start of day	0	8:00 AM	480
12:00 PM	720	2:00 PM	840
3:00 PM	900	4:00 PM	960
5:00 PM	1020	End of day	1440

Cross-midnight limitation
One row cannot cross midnight because END_MINUTE must be greater than START_MINUTE and the day is stored on the row. Split an overnight window into two rows on adjacent days, or extend the data model if overnight windows are required.

Overlap example

Wednesday 8 AM-5 PM available plus Wednesday 2 PM-4 PM unavailable: 1:00-2:00 PM qualifies; 1:30-2:30 PM is rejected; 4:00-5:00 PM qualifies. Touching a blocked interval boundary is not an overlap under [start, end) rules.

3. Existing parent-table prerequisites

The SQL below assumes an existing parent table named INTERVIEWER_MASTER. Replace that name with the actual table, and ensure the child identifier data types and lengths match the parent columns.

3.1 Check for duplicate composite keys

Run this query first. It should return no rows before adding the composite unique constraint.

```
SELECT INTERVIEWER_SID,
       ORACLE_CAND_ID,
       COUNT(*) AS ROW_COUNT
FROM INTERVIEWER_MASTER
GROUP BY INTERVIEWER_SID, ORACLE_CAND_ID
HAVING COUNT(*) > 1;
```

3.2 Ensure the composite key is unique

Run only if an equivalent primary key or unique constraint does not already exist.

```
ALTER TABLE INTERVIEWER_MASTER
ADD CONSTRAINT UQ_INTERVIEWER_SID_CAND
UNIQUE (INTERVIEWER_SID, ORACLE_CAND_ID);
```

3.3 Add a preference activation flag

Run only if the existing table does not already have an equivalent field. The flag distinguishes disabled preferences from an enabled-but-missing rule set.

```
ALTER TABLE INTERVIEWER_MASTER
ADD PREFERENCE_ENABLED CHAR(1) DEFAULT 'N' NOT NULL;

ALTER TABLE INTERVIEWER_MASTER
ADD CONSTRAINT CK_INTERVIEWER_PREF_ENABLED
CHECK (PREFERENCE_ENABLED IN ('Y', 'N'));
```

3.4 Read the interviewer configuration

```
SELECT INTERVIEWER_SID,
       ORACLE_CAND_ID,
       TIME_ZONE,
       PREFERENCE_ENABLED
FROM INTERVIEWER_MASTER
WHERE INTERVIEWER_SID = :interviewerSid
  AND ORACLE_CAND_ID = :oracleCandId;
```

Parent-row prerequisite
The parent row identified by (INTERVIEWER_SID, ORACLE_CAND_ID) must exist before inserting child preference slots. No parent INSERT is supplied because the complete existing parent schema is unknown.

4. Create the preference slot table

The full script below defines the primary key, composite foreign key, validation constraints, and duplicate interval protection. Review ON DELETE CASCADE against your retention rules before production use.

```
CREATE TABLE INTERVIEWER_PREFERENCE_SLOT (
    SLOT_ID           NUMBER GENERATED BY DEFAULT AS IDENTITY,
    INTERVIEWER_SID   VARCHAR2(100) NOT NULL,
    ORACLE_CAND_ID    VARCHAR2(100) NOT NULL,
    DAY_OF_WEEK       NUMBER(1) NOT NULL,
    START_MINUTE      NUMBER(4) NOT NULL,
    END_MINUTE        NUMBER(4) NOT NULL,
    IS_AVAILABLE      NUMBER(1) NOT NULL,
    CREATED_AT        TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    UPDATED_AT        TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,

    CONSTRAINT PK_INTERVIEWER_PREF_SLOT PRIMARY KEY (SLOT_ID),

    CONSTRAINT FK_INTERVIEWER_PREF_SLOT
        FOREIGN KEY (INTERVIEWER_SID, ORACLE_CAND_ID)
        REFERENCES INTERVIEWER_MASTER
            (INTERVIEWER_SID, ORACLE_CAND_ID)
        ON DELETE CASCADE,

    CONSTRAINT CK_PREF_SLOT_DAY
        CHECK (DAY_OF_WEEK BETWEEN 1 AND 7),
    CONSTRAINT CK_PREF_SLOT_START
        CHECK (START_MINUTE BETWEEN 0 AND 1439),
    CONSTRAINT CK_PREF_SLOT_END
        CHECK (END_MINUTE BETWEEN 1 AND 1440),
    CONSTRAINT CK_PREF_SLOT_RANGE
        CHECK (END_MINUTE > START_MINUTE),
    CONSTRAINT CK_PREF_SLOT_AVAILABLE
        CHECK (IS_AVAILABLE IN (0, 1)),

    CONSTRAINT UQ_PREF_SLOT UNIQUE
        (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
         START_MINUTE, END_MINUTE)
);
```

Optional UPDATED_AT trigger

A column default sets UPDATED_AT on insert but does not refresh it on update. Use this optional trigger if the application does not set the value itself.

```
CREATE OR REPLACE TRIGGER TRG_INTERVIEWER_PREF_SLOT_BU
BEFORE UPDATE ON INTERVIEWER_PREFERENCE_SLOT
FOR EACH ROW
BEGIN
    :NEW.UPDATED_AT := SYSTIMESTAMP;
END;
/
```

Index note
The unique constraint normally creates a supporting index whose leading columns include the lookup fields. Start without a separate lookup index; add one only if production query plans show it is needed.

5. Test data - Use case A

Requirement: Monday-Wednesday have no allowed intervals; Thursday 3 PM-5 PM is available; Friday 8 AM-5 PM is available. Because preferences are an allowlist, Monday-Wednesday need no rows.

Sample prerequisites
Parent mappings SID123 / CAND456 and SID789 / CAND999 must already exist in INTERVIEWER_MASTER. These are sample IDs only.

5.1 Insert Thursday and Friday windows

```
INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID123', 'CAND456', 4, 900, 1020, 1);

INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID123', 'CAND456', 5, 480, 1020, 1);

UPDATE INTERVIEWER_MASTER
SET PREFERENCE_ENABLED = 'Y'
WHERE INTERVIEWER_SID = 'SID123'
  AND ORACLE_CAND_ID = 'CAND456';

COMMIT;
```

Expected schedule

Day	Preference interpretation
Monday	No allowed interval
Tuesday	No allowed interval
Wednesday	No allowed interval
Thursday	3 PM-5 PM
Friday	8 AM-5 PM

Example requested intervals

Day and interval	Duration	Result	Reason
Thursday 3:00-4:00 PM	60 min	PASS	Fully inside 3 PM-5 PM.
Thursday 2:30-3:30 PM	60 min	REJECT	Part of the interval is outside the allowed window.
Friday 10:00-11:00 AM	60 min	PASS	Fully inside 8 AM-5 PM.
Monday 10:00-11:00 AM	60 min	REJECT	No allowed interval exists for Monday.

6. Test data - Use case B

Requirement: Monday-Friday 8 AM-5 PM, except Wednesday 2 PM-4 PM is unavailable. A blocked interval overrides an overlapping allowed interval.

```
-- Monday to Friday: 8 AM to 5 PM
INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID789', 'CAND999', 1, 480, 1020, 1);

INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID789', 'CAND999', 2, 480, 1020, 1);

INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID789', 'CAND999', 3, 480, 1020, 1);

INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID789', 'CAND999', 4, 480, 1020, 1);

INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID789', 'CAND999', 5, 480, 1020, 1);

-- Wednesday 2 PM-4 PM unavailable
INSERT INTO INTERVIEWER_PREFERENCE_SLOT
    (INTERVIEWER_SID, ORACLE_CAND_ID, DAY_OF_WEEK,
     START_MINUTE, END_MINUTE, IS_AVAILABLE)
VALUES ('SID789', 'CAND999', 3, 840, 960, 0);

UPDATE INTERVIEWER_MASTER
SET PREFERENCE_ENABLED = 'Y'
WHERE INTERVIEWER_SID = 'SID789'
  AND ORACLE_CAND_ID = 'CAND999';

COMMIT;
```

Expected Wednesday outcomes

Requested interval	Result	Reason
1:00-2:00 PM	PASS	Ends exactly when blocked interval begins; boundary is end-exclusive.
1:30-2:30 PM	REJECT	Overlaps the blocked interval.
2:00-3:00 PM	REJECT	Entirely within blocked interval.
4:00-5:00 PM	PASS	Starts exactly when blocked interval ends.

Test-data caution
Use case A and use case B are independent examples. Do not insert both schedules for the same interviewer mapping unless you intentionally want to combine the rules.

7. SELECT queries and interval evaluation

7.1 Read saved preference rows

```
SELECT DAY_OF_WEEK,
       START_MINUTE,
       END_MINUTE,
       IS_AVAILABLE
FROM INTERVIEWER_PREFERENCE_SLOT
WHERE INTERVIEWER_SID = :interviewerSid
  AND ORACLE_CAND_ID = :oracleCandId
ORDER BY DAY_OF_WEEK, START_MINUTE;
```

7.2 Count saved rows

```
SELECT COUNT(*) AS SLOT_COUNT
FROM INTERVIEWER_PREFERENCE_SLOT
WHERE INTERVIEWER_SID = :interviewerSid
  AND ORACLE_CAND_ID = :oracleCandId;
```

When PREFERENCE_ENABLED = Y and SLOT_COUNT = 0, treat this as an inconsistent preference configuration.

7.3 Check one requested interval

The requested interval must fit wholly within one AVAILABLE window and must not overlap any UNAVAILABLE interval.

```
SELECT CASE
         WHEN EXISTS (
             SELECT 1
             FROM INTERVIEWER_PREFERENCE_SLOT p
             WHERE p.INTERVIEWER_SID = :interviewerSid
               AND p.ORACLE_CAND_ID = :oracleCandId
               AND p.DAY_OF_WEEK = :dayOfWeek
               AND p.IS_AVAILABLE = 1
               AND p.START_MINUTE <= :requestedStartMinute
               AND p.END_MINUTE >= :requestedEndMinute
         )
         AND NOT EXISTS (
             SELECT 1
             FROM INTERVIEWER_PREFERENCE_SLOT p
             WHERE p.INTERVIEWER_SID = :interviewerSid
               AND p.ORACLE_CAND_ID = :oracleCandId
               AND p.DAY_OF_WEEK = :dayOfWeek
               AND p.IS_AVAILABLE = 0
               AND p.START_MINUTE < :requestedEndMinute
               AND p.END_MINUTE > :requestedStartMinute
         )
         THEN 'PASS'
         ELSE 'REJECT'
       END AS PREFERENCE_RESULT
FROM DUAL;
```

7.4 Inspect stored test data

```
SELECT SLOT_ID, INTERVIEWER_SID, ORACLE_CAND_ID,
       DAY_OF_WEEK, START_MINUTE, END_MINUTE, IS_AVAILABLE,
       CREATED_AT, UPDATED_AT
FROM INTERVIEWER_PREFERENCE_SLOT
WHERE INTERVIEWER_SID IN ('SID123', 'SID789')
ORDER BY INTERVIEWER_SID, ORACLE_CAND_ID,
         DAY_OF_WEEK, START_MINUTE;
```

8. Duration-based slot generation

When the candidate selects a date and interview type, Java gets the duration in minutes and the selected date expressed in the interviewer time zone. The database returns start/end minute pairs that meet the weekly preference rules.

8.1 Duration versus start-time step

Duration is how long the interview runs. Step is how far to move forward to get the next candidate start time. For the examples below, set step equal to duration. Keep these runtime values separate so a different cadence can be supported later (for example, a 60-minute interview starting every 30 minutes).

Duration	Step	Valid starts for an 8 AM-5 PM window	Last interval
30 min	30 min	8:00, 8:30, 9:00, ... 4:30 PM	4:30-5:00 PM
45 min	45 min	8:00, 8:45, 9:30, ... 4:15 PM	4:15-5:00 PM
60 min	60 min	8:00, 9:00, 10:00, ... 4:00 PM	4:00-5:00 PM
120 min	120 min	8:00, 10:00, 12:00, 2:00 PM	2:00-4:00 PM

End boundary
A start is valid only when start + duration <= the end of an allowed window. If the window ends at 5:00 PM, then 5:00 PM is not a valid start for any positive duration.

8.2 Window and blocked exception

For an 8 AM-5 PM allowed window with Wednesday 2 PM-4 PM blocked, candidate starts are generated on the configured step. Any generated interval that touches or overlaps the blocked window is removed if there is a real overlap. For example, a 60-minute interview at 1:00-2:00 PM may qualify; 1:30-2:30 PM must be rejected.

8.3 Expected minute values

Duration / step	Candidate start minute values
30 / 30	480, 510, 540, ... 990
45 / 45	480, 525, 570, ... 975
60 / 60	480, 540, 600, ... 960
120 / 120	480, 600, 720, 840

9. Oracle query - generate personalised candidate starts

Bind the interviewer mapping, selected weekday, interview duration, and start-time step. The query generates starts from each AVAILABLE window, rejects intervals extending past that window, and removes any interval overlapping a blocked rule.

```
-- Binds: :interviewerSid, :oracleCandId, :dayOfWeek,
--        :durationMinutes, :slotStepMinutes
WITH slot_offsets AS (
    SELECT LEVEL - 1 AS N
    FROM DUAL
    CONNECT BY LEVEL <= CEIL(1440 / :slotStepMinutes)
), candidate_slots AS (
    SELECT p.INTERVIEWER_SID,
           p.ORACLE_CAND_ID,
           p.DAY_OF_WEEK,
           p.START_MINUTE + (o.N * :slotStepMinutes) AS CANDIDATE_START,
           p.START_MINUTE + (o.N * :slotStepMinutes)
               + :durationMinutes AS CANDIDATE_END
    FROM INTERVIEWER_PREFERENCE_SLOT p
    CROSS JOIN slot_offsets o
    WHERE p.INTERVIEWER_SID = :interviewerSid
      AND p.ORACLE_CAND_ID = :oracleCandId
      AND p.DAY_OF_WEEK = :dayOfWeek
      AND p.IS_AVAILABLE = 1
      AND p.START_MINUTE + (o.N * :slotStepMinutes)
              + :durationMinutes <= p.END_MINUTE
), eligible_slots AS (
    SELECT DISTINCT c.CANDIDATE_START,
                    c.CANDIDATE_END
    FROM candidate_slots c
    WHERE NOT EXISTS (
        SELECT 1
        FROM INTERVIEWER_PREFERENCE_SLOT b
        WHERE b.INTERVIEWER_SID = :interviewerSid
          AND b.ORACLE_CAND_ID = :oracleCandId
          AND b.DAY_OF_WEEK = :dayOfWeek
          AND b.IS_AVAILABLE = 0
          AND b.START_MINUTE < c.CANDIDATE_END
          AND b.END_MINUTE > c.CANDIDATE_START
    )
)
SELECT CANDIDATE_START,
       CANDIDATE_END
FROM eligible_slots
ORDER BY CANDIDATE_START;
```

Bind validation in Java

Validate durationMinutes > 0 and slotStepMinutes > 0, and reject values greater than 1440. Typical durations are 30, 45, 60, and 120 minutes. If the product should offer slots every 30 minutes for a 60-minute interview, pass durationMinutes=60 and slotStepMinutes=30.

Query output
The query returns minute-of-day values. Convert each result into a local date/time using the selected date and the interviewer TIME_ZONE before rendering the start times in the UI.

10. Java-side processing and test checklist

10.1 Suggested Java flow

```
int durationMinutes = request.getDurationMinutes();
int stepMinutes = request.getSlotStepMinutes() != null
        ? request.getSlotStepMinutes()
        : durationMinutes;

validateDurationAndStep(durationMinutes, stepMinutes);

// Use the interviewer's local date, weekday, and time zone.
int dayOfWeek = selectedDate.getDayOfWeek().getValue();

List<PreferenceInterval> eligible = preferenceRepository
        .findCandidateStarts(interviewerSid, oracleCandId,
                dayOfWeek, durationMinutes, stepMinutes);

// Convert minute-of-day results to local date/time values in TIME_ZONE.
// Return start and end times to the booking-calendar UI.
return eligible.stream()
        .map(slot -> toLocalSlot(selectedDate, interviewerTimeZone, slot))
        .toList();
```

10.2 Preference test cases

Test	Input interval	Expected result
Case A: Thursday allowed	900-960 (3-4 PM)	PASS
Case A: starts before window	870-930 (2:30-3:30 PM)	REJECT
Case A: Friday allowed	600-660 (10-11 AM)	PASS
Case A: Monday has no allowed row	600-660 (10-11 AM)	REJECT
Case B: before blocked time	780-840 (1-2 PM)	PASS
Case B: overlaps blocked time	810-870 (1:30-2:30 PM)	REJECT
Case B: inside blocked time	840-900 (2-3 PM)	REJECT
Case B: after blocked time	960-1020 (4-5 PM)	PASS

10.3 Operational checklist

- Save preference changes transactionally: replace the slot rows and set PREFERENCE_ENABLED = Y only after validation succeeds.
- To disable personalisation, clear the rows and set PREFERENCE_ENABLED = N in the same transaction, if that is the chosen product behaviour.
- Re-read preferences before accepting the selected interval, because the interviewer may have changed their schedule.
- Return no personalised intervals when the requested interview does not fit a permitted window.
- Make weekdays consistent with Java DayOfWeek: Monday=1 through Sunday=7.
- Evaluate the recurring weekday and local time in the interviewer time zone, including daylight-saving transitions where relevant.

11. Final design notes

Identifier relationship

The foreign key assumes (INTERVIEWER_SID, ORACLE_CAND_ID) identifies a single interviewer mapping in INTERVIEWER_MASTER. If ORACLE_CAND_ID actually identifies the candidate being interviewed rather than the interviewer, preferences should normally be keyed by interviewer SID (or another stable interviewer identifier) instead of candidate.

Availability windows and exceptions

Allowed windows define the time range from which candidate slots are generated. Blocked windows remove any generated interval with a real overlap. A requested interval must fit fully within one allowed window; it cannot straddle a gap between separate allowed windows.

Start-time cadence

The default step can equal the duration to match the requested examples. Keep duration and step as separate request values so the UI can later offer staggered starts without a schema change. The SQL query aligns each cadence to the start of each allowed window.

What this document covers

This guide covers the personalised preference data model, Oracle DDL, test examples, preference-check SQL, and duration-based generation of start/end intervals. It intentionally does not define external availability integrations or the booking persistence implementation.

Implementation summary
One new slot table is enough for recurring weekly windows and blocked exceptions. The preference query returns only intervals that pass the allowlist and overlap rules. The UI renders the returned local start times for the selected date.

Companion file: interviewer_personalised_availability.sql contains the DDL, test inserts, SELECT queries, single-interval preference check, and duration-based candidate-start query.